



University of Southern Brittany – UBS

Course Name:

M1-ISC-CSSE-CYBERUS - Applied Cryptography

\$

Lecturer: Professor Guy GOGNIAT

LAB REPORT – 3 Part 2
(Cryptography Protocol)

Submitted by :

Name; Assan Jallow

Student ID: E2506147

Email: jallow.e2506147@etud.univ-ubs.fr

Lab 3: Part-2 Cryptography Protocol

1. Introduction

This report presents the implementation and testing of three cryptographic protocols developed in Python as part of *Lab 3: Part 2 – Cryptographic Protocols*.

The aim of the lab was to apply both asymmetric (public key) and symmetric (private key) cryptographic techniques to design secure communication between two parties, traditionally called *Alice* and *Bob*.

Each protocol demonstrates a critical cryptographic principle:

- Confidentiality (through encryption)
- Integrity and authenticity (through digital signatures and MACs)
- Key exchange (through Diffie–Hellman)

To achieve this, I developed Python scripts using the PyCryptodome library and additional helper scripts to manage encryption, decryption, and file operations.

In addition to the three required protocols, I created a manual implementation using my own AES algorithm from Lab 2, integrating it into the key transport protocol to demonstrate how low-level cryptographic building blocks combine to form complete systems.

2. Project Structure and Files

All implementations are organized into a single folder named `lab3_part2_crypto_protocols`, which contains both code and generated test artifacts.

Main Python Files

1. `common.py`

This file contains shared helper functions used by all the protocols.

It includes AES-GCM encryption and decryption routines, HKDF key derivation, and utilities for reading/writing files.

This modular design helped avoid code repetition across multiple scripts.

2. `protocol1_key_transport_pycrypto.py`

Implements the Basic Key Transport Protocol using RSA-2048 and AES-128 (in GCM mode).

It simulates both Alice and Bob in one script. Alice generates a random AES session key,

encrypts a plaintext file, and then encrypts the session key using Bob's RSA public key. Bob decrypts the session key using his private key and recovers the original message. The results, including ciphertext, keys, and recovered plaintext, are stored in artifacts/protocol1/.

3. protocol2_rsa_signature.py

Implements the RSA Digital Signature Protocol.

Alice signs a message using RSA-PSS with SHA-256, and Bob verifies the signature using Alice's public key.

I confirmed correctness by tampering with the signature and verifying that it failed, which showed integrity protection was working.

The artifacts (message, signature, and keys) are stored in artifacts/protocol2/.

4. protocol3_diffie_hellman_aes.py

Implements the Diffie–Hellman Key Exchange Protocol with AES-128 encryption.

Both Alice and Bob generate their DH key pairs, exchange public values, and compute the shared secret.

The secret is then passed through HKDF-SHA256 to derive an AES key, which is used to encrypt and decrypt a test message with AES-GCM.

Successful decryption confirmed that both sides derived identical keys.

Outputs are saved in artifacts/protocol3/.

5. protocol1_key_transport_manual_TEMPLATE.py

This is the manual version of Protocol 1.

It was created to integrate the AES algorithm I implemented in Lab 2, replacing the PyCryptodome AES functions with my own block cipher.

It uses RSA operations implemented manually with $\text{pow}(m, e, n)$ and PKCS#1 v1.5 padding.

I also added HMAC-SHA256 for integrity (Encrypt-then-MAC).

This version demonstrates a complete protocol using only fundamental primitives — a good exercise for understanding what modern libraries abstract away.

6. lab2_aes.py

This file contains my AES algorithm from Lab 2, adapted to be callable within the manual protocol.

It defines the S-box, key schedule, SubBytes, ShiftRows, MixColumns, and AddRoundKey functions, forming the AES encryption process for a 16-byte block.

3. Implementation Details

Protocol 1 – RSA-OAEP and AES-GCM (Key Transport)

I started with the hybrid encryption approach:

1. Bob generates RSA-2048 keys and shares his public key.
2. Alice generates a random AES-128 session key and encrypts her plaintext using AES-GCM.
3. She then encrypts the session key with RSA-OAEP and sends both to Bob.
4. Bob decrypts the session key using his private key, then uses it to decrypt the AES-GCM ciphertext.

The protocol demonstrates hybrid cryptography: using RSA for secure key transport and AES for fast bulk encryption.

All intermediate outputs, including encrypted keys, ciphertext, and authentication tags, were verified and stored.

Tampering with the ciphertext caused the AES-GCM verification to fail, confirming that the integrity checks worked.

Protocol 2 – RSA Digital Signature

For this protocol, I used RSA-PSS with SHA-256 to produce secure digital signatures.

- Alice generates a 2048-bit RSA key pair.
- She signs the hash of her message using the private key and sends both the message and signature to Bob.
- Bob verifies the signature using Alice's public key.

To test this, I manually altered the message and the signature files. Both modifications resulted in verification failure, proving that even a single-bit change is detected.

This exercise clearly demonstrated authenticity (message came from Alice) and integrity (message unchanged).

Protocol 3 – Diffie–Hellman Key Exchange

In this part, I implemented a 2048-bit Diffie–Hellman key exchange based on the MODP group parameters from RFC 3526.

Both Alice and Bob independently generated private values a and b , computed $A = g^a \bmod p$ and $B = g^b \bmod p$, and exchanged these public values.

The shared secret $s = g^{(ab)} \bmod p$ was then derived identically by both.

I used HKDF-SHA256 to derive a uniform 128-bit AES key from this shared secret.

This key was then used for AES-GCM encryption and decryption.

Decryption succeeded only when both sides had matching secrets, verifying the correctness of the implementation.

Manual Extension – Protocol 1 using Lab 2 AES

The manual implementation extended the first protocol to use my own AES algorithm developed in Lab 2.

In this version:

- The AES encryption used a simple CTR mode built on top of my AES block cipher.
- RSA encryption/decryption was done manually using the public and private key exponents with `pow()` and PKCS#1 v1.5 padding.
- I added HMAC-SHA256 to ensure message integrity, making it an *Encrypt-then-MAC* scheme.
Although slower than the PyCryptodome version, it worked correctly once I connected my AES function.

4. Testing and Validation

Each protocol was tested independently using small plaintext files (plaintext.txt) to confirm correct encryption/decryption or signature verification.

All keys and messages were written to the artifacts folder for inspection.

- For Protocol 1 and 3, I verified that decrypted text matched the original input.
- For Protocol 2, signature verification failed upon tampering, as expected.
- For the manual version, I verified both confidentiality and integrity (HMAC mismatch detection).